

High-throughput JSONPath Query Execution on Neural Processing Units

Hexiang Geng

Vrije Universiteit Amsterdam
h.g.geng@vu.nl

Tiziano De Matteis

Vrije Universiteit Amsterdam
t.de.matteis@vu.nl

Jorin Tielen

Eindhoven University of Technology
j.h.tielen@student.tue.nl

Daniele Bonetta

Vrije Universiteit Amsterdam
d.bonetta@vu.nl

ABSTRACT

JSON is the de facto interchange format for a wide variety of domains, making JSON processing a central component of many data-intensive applications. As data volumes grow, parsing and querying JSON increasingly represent a critical performance bottleneck.

Recent SIMD-optimized CPU libraries have shown that JSON can be processed at GB/s throughput, enabling high-performance JSON analytics on general-purpose processors. Meanwhile, new on-chip AI accelerators such as Neural Processing Units (NPUs) provide high-throughput vector computation that could also potentially benefit JSON processing. Yet, their role in general-purpose data processing remains largely unexplored.

In this paper, we present **NPJ-JSON**, the first JSONPath query engine accelerated by an NPU. Our system uses a hybrid architecture that offloads the most computationally intensive stages of JSON parsing to an AMD XDNA NPU, while the host CPU concurrently executes JSONPath queries. Our design maximizes hardware utilization through software pipelining and optimized memory management between the CPU and NPU. We evaluate NPJ-JSON on two generations of AMD XDNA NPUs against state-of-the-art CPU-based and GPU-based query engines across a wide range of datasets and queries. Our results show highly competitive performance, achieving a throughput of up to 9 GB/s and outperforming existing systems. This work provides the first empirical evidence that NPUs can serve as powerful general-purpose coprocessors for data analytics, opening new usage scenarios beyond their primary AI/ML domain.

PVLDB Reference Format:

Hexiang Geng, Jorin Tielen, Tiziano De Matteis, and Daniele Bonetta.
High-throughput JSONPath Query Execution on Neural Processing Units.
PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/npj-json-vldb/npj-json>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

The JavaScript Object Notation (JSON) format [29] is the de facto standard for data interchange across web services, cloud applications, and data analytics pipelines. Its simplicity and human readability have led to widespread adoption in domains ranging from real-time logging and monitoring [17] to data lakes [16]. The impact of JSON processing on system performance is substantial: ByteDance reported that their servers spend on average 10% of execution time parsing JSON, with the overhead reaching up to 40% in extreme cases [14]. JSON parsing is also important for client-side applications, where it affects the latency and responsiveness of processing data received from Web services [10, 32, 35]. JSON adoption is not slowing down, and the data format is natively supported by most modern database systems (e.g., PostgreSQL [40], DuckDB [42]) and widely used in data science formats such as GeoJSON [13]. Given the predominance of JSON, efficient tools to perform high-throughput JSON parsing and querying are critical to ensure fast data access and low-latency analytics.

In recent years, significant research has shown that exploiting hardware parallelism can dramatically improve JSON analytics performance. Systems such as *simdjson* [34], *JSONSki* [28], and *rsonpath* [21] leverage SIMD vector instructions to accelerate both parsing and JSONPath [23] query evaluation, achieving gigabytes per second throughput. At the same time, modern processors increasingly include heterogeneous accelerators on the same chip, including integrated GPUs and *Neural Processing Units* (NPUs). While GPUs have evolved from graphics engines into widely used platforms for data analytics [20, 47, 48], on-chip NPUs remain largely unused by data processing systems.

In this paper, we present **NPJ-JSON**, the first JSONPath query engine accelerated by an AMD XDNA NPU [25, 44]. By offloading the most computationally intensive stage of JSON parsing to the NPU while keeping branch-heavy query execution on the CPU, NPJ-JSON achieves high throughput and efficient hardware utilization. Thanks to its hybrid NPU-CPU architecture, NPJ-JSON can provide competitive or superior performance compared to state-of-the-art CPU-only engines. This paper makes the following contributions:

- We introduce NPJ-JSON, an open-source JSONPath query engine that uses a CPU-NPU streaming architecture to offload structural index construction to the NPU while executing the query automaton on the CPU. To the best of our knowledge, NPJ-JSON is the first JSON analytics engine accelerated by NPU.
- We describe the design of a pipelined strategy with CPU-NPU buffering and memory management that overlaps index-

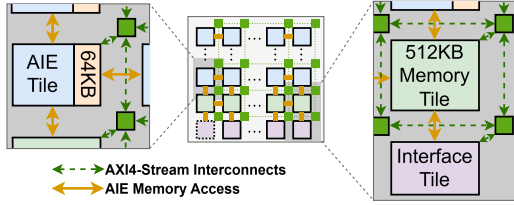


Figure 2: XDNA NPU architecture. The compute tiles contain an AI Engine with a small local memory of 64 KB. Memory tiles have 512 KB each [51]. On some NPU configurations, column 0 does not have an interface tile.

using either low-level intrinsics or higher-level APIs [52]. The communication structure between kernels is then defined as a dataflow graph expressed using the Python-based IRON DSL [25]. The graph specifies how kernels are placed onto tiles, how tiles are interconnected, the sizes of intermediate buffers, and the movement of data between them. The resulting configuration determines the execution topology of the application across the NPU. A host-side C/C++ program interacts with the accelerator through the Xilinx Runtime Library (XRT) [5], which manages device initialization, memory transfers, and kernel execution. Communication across the accelerator is implemented through a streaming network-on-chip that supports point-to-point transfers as well as broadcast and selective multicast patterns. Unlike architectures that rely on hardware-managed caches (e.g., CPUs), the NPU uses explicitly managed scratchpad memories. Data movement is therefore controlled in software and follows deterministic schedules. Dedicated DMA engines orchestrate asynchronous transfers between memories and tiles. Using programmable buffer descriptors, applications can implement operations such as reordering, reshaping, and repeating data streams.

Challenges for Data Processing. While SIMD-based techniques have revolutionized JSON parsing on CPUs [34], the data-parallel structure of index construction makes it well suited to other specialized vector hardware. However, SIMD-based techniques cannot be directly ported to NPUs due to the differences in execution model, memory hierarchy, and programming interface.

First, XDNA NPUs are optimized for vector-heavy workloads and achieve high throughput when executing regular data-parallel computations. However, they perform very poorly on scalar logic with complex control flow compared to modern CPUs, as they lack sophisticated branch prediction and out-of-order execution mechanisms. In JSONPath processing, this distinction is critical. Low-level parsing stages, such as character classification or structural indexing, can benefit from vectorization. In contrast, evaluating a JSONPath automaton is inherently stateful and branch-intensive, involving frequent control-flow decisions and irregular memory access patterns. As a result, workload distribution between the CPU and the NPU must be carefully designed, assigning vector-friendly operations to the accelerator while retaining state management and control-heavy logic on the CPU.

Second, the strict dataflow execution model and the absence of shared, hardware-managed caches require algorithms to be reformulated into a tiled, streaming-oriented paradigm. Buffer sizes for

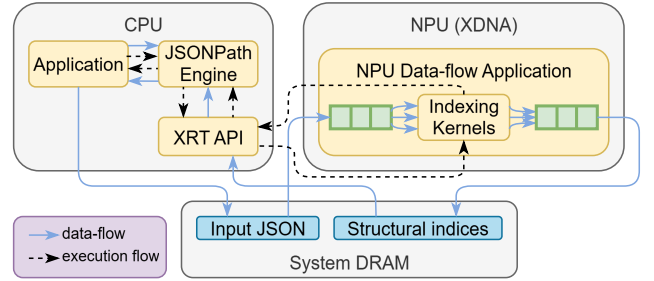


Figure 3: NPU-JSON high-level architecture.

transfers between the host and the NPU, as well as for internal data movement, must be explicitly defined and are typically fixed at compile time. This requirement conflicts with the inherently dynamic and variable structure of JSON documents, where token boundaries and structural depth are not known in advance. Consequently, the input must be partitioned into manageable chunks, and algorithms must be designed to operate correctly under streaming constraints while preserving parsing correctness across chunk boundaries. This requirement also distinguishes the XDNA NPUs from other acceleration targets. Existing high-performance JSON parsers for SIMD-enabled CPUs [34, 35] or GPUs [20, 47] typically assume large memory spaces where an entire JSON document (or large parts of it) can be loaded. These algorithms rely on loading large memory regions, performing vectorized scans, and writing results back to memory without explicitly managing the movement of data between processing units. On the XDNA NPU, however, both the partitioning of the input and the movement of data between host memory, on-chip scratchpads, and compute tiles must be explicitly orchestrated. The programmer must therefore define the size and layout of streaming buffers, the scheduling of DMA transfers, and the topology of the dataflow graph. As a result, algorithms designed for SIMD or GPUs cannot be ported directly: they must be re-designed in light of the streaming, tile-based execution model of the NPU.

3 NPU-JSON

Our goal is to use on-chip NPUs to accelerate JSONPath query processing. To this end, NPU-JSON was designed to balance NPU offloading with CPU coordination and data movement to sustain high throughput. From a user’s perspective, NPU-JSON is a JSONPath C++ library. It exposes the same APIs as equivalent JSON analytics libraries (e.g., [21]), while internally combining parsing on the NPU with CPU-based query execution. Figure 3 shows the high-level architecture of the library and its runtime: the JSON input is indexed in parallel on the NPU, and then consumed by a JSONPath query engine running on the CPU. This hybrid architecture enables overlap between CPU and NPU computation, hiding communication costs, and maximizing hardware utilization.

Data processing in NPU-JSON is achieved in three distinct stages, namely: (1) data partitioning, (2) indexing, and (3) query execution. When a query is first submitted, the input JSON is partitioned on the CPU into chunks that can be processed independently. For each of them, the NPU constructs structural indexes that identify relevant symbols in the input data (string and structural characters).

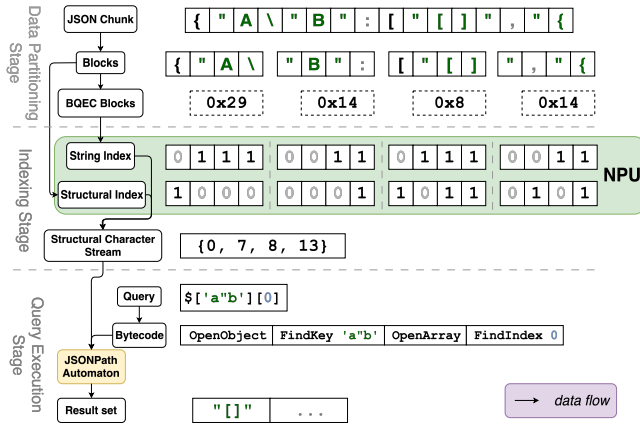


Figure 4: Overview of NPU-JSON’s data processing stages.

Once such symbols have been identified, they are streamed back to the CPU, which then executes the JSONPath query using the indexes computed on the NPU. By keeping branch-heavy query automaton execution on the CPU while offloading parallel parsing to the NPU, NPU-JSON balances the strengths of both processors. An overview of the three-stage data processing pipeline of NPU-JSON is given in Figure 4. We discuss each step individually in the following sections.

3.1 Data Partitioning Stage

JSON documents can be of arbitrary size, yet the NPU programming model requires buffer sizes to be fixed at compile time. Moreover, kernels running on the NPU do not have access to DRAM and have very limited local memory (see Figure 2). These constraints make it infeasible to index an entire JSON document within a single NPU kernel call. To address this, NPU-JSON partitions the input into fixed-size *chunks* and streams them from the CPU to the NPU. Chunks have configurable size (e.g., 8 MB), but typically do not fit in the very limited memory of an NPU AIE compute tile. To enable data parallelism, NPU-JSON further partitions each chunk into multiple *blocks* with a size that fits in the NPU tile. An overview of the data partitioning and pre-processing steps performed by NPU-JSON is depicted in Figure 5.

Data partitioning provides two key benefits. First, it enables the processing of inputs larger than DRAM. Second, it allows for pipelined execution between CPU and NPU: while the CPU operates on chunk i , the NPU can already begin indexing chunk $i + 1$. This overlap is crucial to maximize utilization of the NPU and reduce end-to-end query latency.

While data partitioning improves performance, it complicates correct JSON parsing. Since JSON string boundaries and escape sequences may span multiple blocks, care must be taken to ensure that block-level independence does not compromise correctness. To address this, NPU-JSON pre-computes some lightweight metadata while performing data partitioning. Such pre-computed parsing metadata will be used by the NPU to correctly parse each individual block in parallel. The metadata consists of the following three indexes:

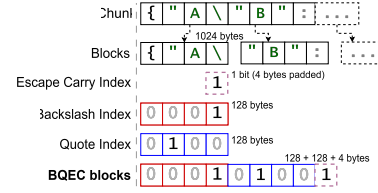


Figure 5: Data partitioning in chunks and blocks, with corresponding indexes and BQEC blocks for the first block. The size of each block in the figures is fictional, while the real size is reported next to each block (e.g., 1024 bytes).

- **Escape carry index:** For each block, one bit is set to signal that the block terminates with an odd sequence of backslash characters. This means that the subsequent block should be parsed as an escaped character sequence.
- **Quote index:** An index where a bit is set for each quote (") character in the block.
- **Backslash index:** Like the quote index, but for backslash (\) characters.

Once these three indexes are computed, they are assembled (on the CPU) into a dedicated block called *BQEC* block, and stored in a contiguous buffer. The buffer contains one BQEC metadata block for each input JSON block. The input JSON blocks and the BQEC blocks are then sent to the NPU, which will perform the remaining compute-heavy parsing operations in the next stage.

3.2 Index Construction Stage

After data have been partitioned on the CPU, the NPU performs the most compute-intensive stage: building structural indexes. This is achieved using two main kernels:

- **String index kernel:** locates all valid strings in the JSON document, making sure that escaped symbols are handled correctly.
- **Structural index kernel:** locates structural characters (`{`, `}`, `[`, `]`, `:`, `,`) ignoring them when appearing within strings.

Both kernels are executed in parallel on the NPU using multiple AIE tiles. The NPU programming model provides flexibility in how data flows are configured, but this requires careful design to fully utilize the hardware. NPU-JSON specifies its data-flow model for each NPU column. In this model, JSON and BQEC blocks are the transfer unit of the NPU FIFO queues: all blocks of a chunk are produced at the interface tile and, through a broadcast operation at the memory tile, are distributed to the compute tiles. Each compute tile processes a single block per kernel invocation and writes the results back via the output FIFOs. To compute both indexes in parallel, two kernels are executed within each column, divided evenly across the available compute tiles. Figure 6 illustrates the dataflow architecture of an NPU column in NPU-JSON: for each kernel, two sets of FIFOs connect the interface and memory tiles. Each set holds two blocks as FIFO units, which the memory tile splits to one per compute tile. This per-column design allows NPU-JSON to scale up to any number of columns and (even-numbered) rows of compute tiles, and each column can work on its own blocks independently. As discussed, blocks are processed in parallel but still carry data dependencies between them. One such dependency is the escape carry: if a block ends with an odd number of backslashes,

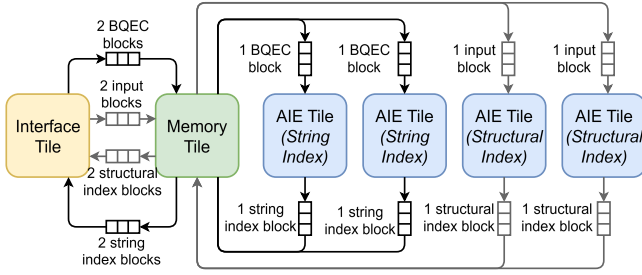


Figure 6: NPU column data-flow, laid out horizontally. Two compute tiles are dedicated to the string index, while the other two perform the structural index.

the first character of the following block must be treated as escaped. To handle this, the kernels responsible for building the string index use the metadata contained in the BQEC blocks to correctly handle parsing of escaped sequences. In the following, we detail how both indexes are constructed on the NPU, explaining how the construction differs from existing SIMD-based techniques.

String Index. String indexing has been widely explored in SIMD-based JSON parsers [34], but SIMD-friendly techniques cannot be applied directly to an NPU. Although the NPU provides SIMD-like vector operations, its programming model, memory hierarchy, and communication constraints differ fundamentally from those of CPUs. To address this, NPU-JSON introduces a new string indexing technique tailored specifically to the NPU. The complete algorithm is shown in Figure 7. The code in the figure executes in a loop over SIMD vectors of an input block. Parsing begins with the quote and backslash indexes q and b , which are precomputed in the BQEC block. To identify escaped characters, the algorithm detects odd-length backslash sequences and shifts them right by one. As shown at point ① in the figure, a state variable tracks whether the previous block ended with an escape. This variable is used to fill in the bit at the start of the vector, while the bit shifted out becomes the carry for the next iteration. Then, the quote index q is masked with the inverse of the escaped characters e , producing the set of unescaped quotes uq . These unescaped quotes are then used in a prefix-XOR operation to build the string index. Finally, at point ②, the algorithm updates another state variable if the last bit of the vector is set, meaning the block ends inside a string. In that case, the next iteration inverts the computed string index. Because both x86 CPUs and the NPU are little-endian, the SIMD comparison bitmask starts from the most-significant bit for the first character—opposite to the visualization in Figure 7. Although this technique is comparable to techniques used in prior work [27, 34], several adaptations are required for execution on the NPU. Due to the limited number of output DMA channels on a memory tile, the escape carry flag cannot be transmitted alongside the block data as a separate stream. Instead, NPU-JSON prepares the input buffer so that each block is followed by its carry flag. A direct memory copy of all blocks would be prohibitively expensive, so the algorithm offloads only the compact BQEC bitmaps of quotes and backslashes to the NPU. Since each index represents one byte with a single bit, the buffer is reduced to one-eighth of the original string data, and including both quote and backslash indexes

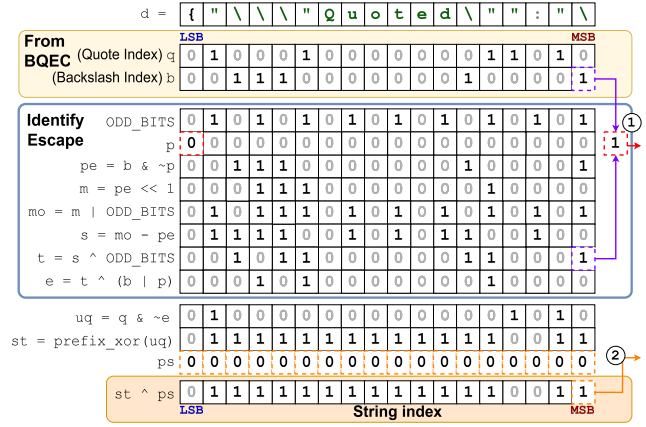


Figure 7: Code sequence implementing the string index, with the resulting data visualized at each step.

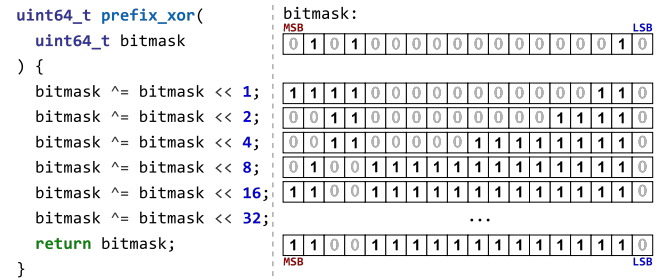


Figure 8: Prefix-XOR function as implemented on NPU and used in Figure 7.

yields a total of one-quarter of the block size. This compact representation reduces memory traffic and accelerates data transfer to the NPU. A final adjustment concerns the prefix-XOR operation. On CPUs, this step can exploit the carry-less multiplication instruction (`pclmulqdq`) [3], which is not available on the NPU. NPU-JSON therefore implements prefix-XOR using a sequence of bit shifts and XOR instructions, achieving the same effect within the NPU’s constraints. This is depicted in Figure 8.

Structural Character Index. In comparison to string indexing, the construction of the structural character index on the NPU is relatively straightforward. The NPU kernel has to match all six structural characters of JSON: opening and closing curly braces, opening and closing square brackets, colons, and commas. In CPU-based SIMD algorithms, specialized instructions (e.g., shuffles) are often used to optimize the matching of such character sets¹. However, the NPU benefits from higher parallelism and has a theoretical maximum throughput of 4 GB/s [51] for a single tile. As long as the kernel remains lightweight, further instruction-level optimizations have little effect. For this reason, the implementation simply performs six vectorized comparisons (one per structural character) and combines the results with a logical OR. The kernel implementation is provided in Listing 1. After retrieving the mask of structural

¹On the NPU, only a limited equivalent is available in the form of `aie::parallel_lookup` [52].

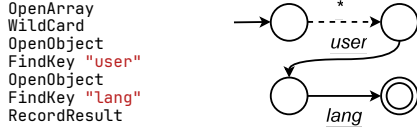


Figure 9: Bytecode for the query $[\ast].\text{user.lang}$ with its corresponding automaton. The striped arrow matches array elements, and the filled arrows match object fields.

characters, NPU-JSON applies the string index to remove structural characters that appear inside strings. The remaining mask is then converted into an array of integer positions, which is streamed to the JSONPath engine for query execution on the CPU.

3.3 Query Execution Stage

JSONPath queries can be implemented with a pushdown automaton, where each state represents a matching condition and transitions encode query progression. In NPU-JSON, this automaton does not operate on raw JSON text but on the structural character stream generated by the NPU.

To support arbitrary queries, NPU-JSON compiles JSONPath expressions into a custom bytecode format that is executed by a lightweight on-CPU interpreter. Each bytecode instruction corresponds to either a state or a transition within the automaton, and advances over the structural data accordingly. Instructions such as FindKey, FindIndex, OpenObject, OpenArray, and RecordResult capture the core steps of JSONPath evaluation, allowing the automaton to traverse the parsed structure and produce query results. The bytecode is a compact sequence of instructions required for query execution. The full instruction set includes:

- OpenObject: find the start of an object,
- OpenArray: find the start of an array,
- FindKey: search for a specific key in an object,
- FindIndex: move to a particular index in an array,
- FindRange: iterate over a range of indexes in an array,
- RecordResult: update the result set.

Each instruction may consume an arbitrary amount of input to complete its task. If an instruction fails, execution returns to the previous instruction. For example, if FindKey locates the target key, the automaton interpreter advances to the next instruction. If it reaches the end of the object without finding the key, execution falls back to the preceding instruction to continue query evaluation. An example query with its automaton and corresponding bytecode is shown in Figure 9.

3.4 Pipelined execution & ping-pong buffering

Data partitioning and the three-stage data processing approach of NPU-JSON allow for both data parallelization on the NPU and pipelined execution between CPU and NPU. Each chunk is first indexed on the NPU. Then, the automaton can use the indexes to continue evaluating the query over each chunk. During this time, the NPU can already start indexing the next chunk, so it might be ready when the CPU automaton finishes processing its current chunk. This effect is shown in Figure 10: overlapping execution can be achieved for all chunks except the first and last ones. This

```
for (size_t i = 0; i < n; i += V) {
    const aie::vector<uint8_t, V> data = aie::load_v<V>(data_ptr);
    data_ptr += V;

    // Match both opening and closing curly braces and combine
    // their index bitmaps.
    auto brace_open = aie::eq(data, brace_open_mask).to_uint64();
    auto brace_close = aie::eq(data, brace_close_mask).to_uint64();
    uint64_t braces = brace_open | brace_close;

    // Match both opening and closing square brackets and combine
    // their index bitmaps.
    auto bracket_open = aie::eq(data, brk_open_mask).to_uint64();
    auto bracket_close = aie::eq(data, brk_close_mask).to_uint64();
    uint64_t brackets = bracket_open | bracket_close;

    // Match both colons and commas and combine their index bitmaps.
    auto colon = aie::eq(data, colon_mask).to_uint64();
    auto comma = aie::eq(data, comma_mask).to_uint64();
    uint64_t colons_and_commas = colon | comma;

    // Combine all three index bitmaps into a bitmap of all
    // structural characters.
    uint64_t structurals = braces | brackets | colons_and_commas;

    // Write structural character bitmap to index.
    *index_buffer++ = structurals;
}
```

Listing 1: Code fragment showing the body of the NPU kernel responsible for computing the structural characters index using XDNA’s AI Engine API. The input data to the NPU compute tile is accessed via the `data_ptr` pointer.

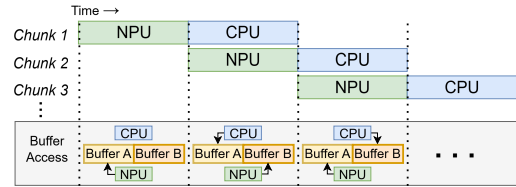


Figure 10: Pipelining and ping-pong (double) buffering. Both NPU and CPU alternate the processed buffer, and each buffer is only accessed by one device at a time.

means a chunk size has to be chosen that is not too large for the size of the dataset. If the dataset is only split into a few chunks, the benefit would be limited.

To maximize NPU utilization, it is important to avoid idle time while the CPU prepares input or consumes output. Since NPU kernels operate on buffers allocated in system DRAM, the CPU must fill these buffers with input data and read back results after execution. During these transfers, the NPU cannot make progress, leading to stalls. To mitigate this overhead, NPU-JSON employs *ping-pong* buffering (a.k.a. double buffering). By overlapping CPU buffer management with NPU computation, we ensure that the NPU can continue processing while the CPU prepares the next batch of data. In our implementation, NPU-JSON manages two sets of input/output buffers, A buffers and B buffers. While the NPU is executing a kernel on the A buffers, the B buffers can be prepared by the CPU. Once the NPU finishes, NPU-JSON can switch between buffers and immediately let the NPU execute the next kernel on buffer B, while reading the results from A, and preparing the next chunk of input data. A visual example of the technique is given in Figure 10. This technique allows the NPU to constantly be utilized, provided the CPU can prepare the buffers faster than the NPU processes them.

3.5 Limitations and Generalizability

Our current implementation supports a large part of the JSONPath language, including member selectors such as `.name`, object and array wildcards such as `.*` and `[*]`, positive array indexes such as `[123]`, positive half-open array ranges such as `[1:3]`, and arbitrary compositions of these operators. Additional JSONPath features, such as descendant queries [21] and function extensions [43] are not supported. Adding support for such features would require changes to the NPU-JSON CPU bytecode interpreter (Section 3.3), but not to the NPU-based index construction.

Another limitation of our current implementation is that a single JSON document can be at most 4 GB. The reason for this limitation is that NPU-JSON represents positions in the structural index using 32-bit offsets. This keeps the structural-index representation compact, reduces memory traffic, and is sufficient for all datasets evaluated in this paper. Not supporting individual documents larger than 4 GB is not a limitation unique to NPU-JSON: widely used SIMD-based parsers such as `simdjson` [34] impose the same practical limit for the same underlying reason, as discussed in [45]. Importantly, this is not a structural limitation of either our architecture or the NPU execution model. Supporting larger individual documents would require adding a 64-bit structural-index representation and updating the related host-side code accordingly. Nothing in the design prevents supporting both variants, for example, by using the current 32-bit representation for documents up to 4GB and a 64-bit representation for larger inputs. We leave the implementation and detailed evaluation of a 64-bit index representation for future work.

Portability to Other NPUs and Accelerators. Our work explicitly targets the AMD XDNA NPU platform. XDNA is particularly well suited to our use case because it provides a low-level, general-purpose programming model that supports custom data-parallel kernels and explicit control over data movement. We evaluate two generations of XDNA, and our performance results apply specifically to this platform family. In principle, the high-level CPU/NPU architecture of NPU-JSON is not inherently tied to XDNA. Some server-grade spatial ML accelerators, including AMD Versal ACAP [19], Tenstorrent Blackhole [49], and the Cerebras Wafer Scale Engine [36], share relevant architectural properties with XDNA, such as vectorized processing elements, distributed local memories, and high-bandwidth on-chip communication. However, our findings do not automatically translate to these server-class accelerators, because differences in scale, memory hierarchy, host-device communication, and programming model can materially affect both the implementation strategy and its performance. Porting our indexing algorithms to other NPUs embedded in commodity CPUs, such as the ones exposed through Intel OpenVINO [26] or Qualcomm QNN [41], presents different challenges. In particular, our string-indexing algorithm described in Section 3.2 relies on prefix scans, packed bitwise operations, explicit data movement, and local-memory management, which are not generally exposed by high-level AI/ML interfaces. A port to these platforms would therefore require reformulating parts of the indexing algorithm and evaluating whether the resulting implementation remains efficient. Exploring both these NPU platforms and server-grade spatial accelerators remains future work.

Table 1: JSONPath queries and datasets used in the experiments. Each dataset is composed of a single JSON object (size in MB), which is replicated 100 times in our experiments to measure maximum throughput.

ID	Dataset (MB)	Query	Results
T1	Twitter (843)	<code>\$[*].user.lang</code>	150135
T2	Twitter (843)	<code>\$[*].entities.urls[*].lang</code>	88881
B1	Best Buy (1045)	<code>\$.products[*].categoryPath[1:3].id</code>	459332
B2	Best Buy (1045)	<code>\$.products[*].videoChapters[*].chapter</code>	697440
G1	Google Maps (1136)	<code>\$[*].routes[*].legs[*].steps[*].distance.text</code>	1716752
G2	Google Maps (1136)	<code>\$[*].available_travel_modes</code>	90
N1	NSPL (1210)	<code>\$.meta.view.columns[*].name</code>	44
N2	NSPL (1210)	<code>\$.data[*][*][*]</code>	8774410
W1	Walmart (995)	<code>\$.items[*].bestMarketplacePrice.price</code>	15892
Wi	Wikipedia (1099)	<code>\$[*].claims.P150[*].mainsnak.property</code>	15603

4 PERFORMANCE EVALUATION

In this section, we evaluate the performance of NPU-JSON on realistic datasets and JSONPath queries, comparing it against other high-performance JSON parsers and analyzing its internal components.

4.1 Hardware Configuration and Methodology

All experiments were conducted using two dedicated machines. The first machine (XDNA1) is a mini-desktop machine equipped with an AMD Ryzen 9 7940HS CPU (Zen 4 “Phoenix”, 8 cores / 16 threads, 4.0 GHz base, 5.3 GHz boost frequency) and an XDNA 1 NPU in a 4-column configuration (16 AIE tiles), along with 32 GB of RAM. The second machine (XDNA2 in the rest of this section) is a laptop machine featuring an AMD Ryzen AI 9 HX 370 CPU (Zen 5 “Strix Point”, 12 cores / 24 threads, 2.0 GHz base, 4.4 GHz boost) and an XDNA 2 NPU with 8 columns (32 AIE tiles), paired with 24 GB of RAM. Having two platforms with different NPU generations (and different numbers of tiles) allows us to assess the performance of our implementation across distinct hardware configurations. Both CPUs support AVX-512, with an architectural difference: Zen 4 implements AVX-512 with a “double pumped” 256-bit datapath, while Zen 5 provides a native 512-bit datapath [33]. Both machines run Ubuntu Linux 24.04 (kernel family 6.11.0). All code was compiled with GCC 13.3.0 under flags `-O3 -DNDEBUG -march=native`, using the IRON/MLIR-AIE toolchain v1.2.1, enabling maximum optimization, removing debug assertions, and targeting the best available SIMD instruction set (AVX-512 on both machines). To avoid networking or disk overheads, the entire input dataset is pre-loaded in DRAM. This is the same approach adopted by the engines we compare against [21, 28, 30]. To measure maximum throughput, we stream the same JSON object to the query engine 100 times for each query, resulting in each experiment processing roughly 90–120 GB of JSON data². This allows the engine to reach steady-state behavior and saturate its processing pipeline, which enables reliable GB/s throughput measurements. We additionally perform 25 warm-up runs beforehand to warm up CPU caches and stabilize execution. In Section 4.3.6 we also discuss the cold-start performance of NPU-JSON. For all our experiments, we always set the CPU clock rate to its maximum frequency, bind threads to the best-performing core (e.g., “big” cores on Zen 5), and

²The size of each JSON document is reported in Table 1. The exact amount of data for each experiment is computed by multiplying the size of each object by 100.

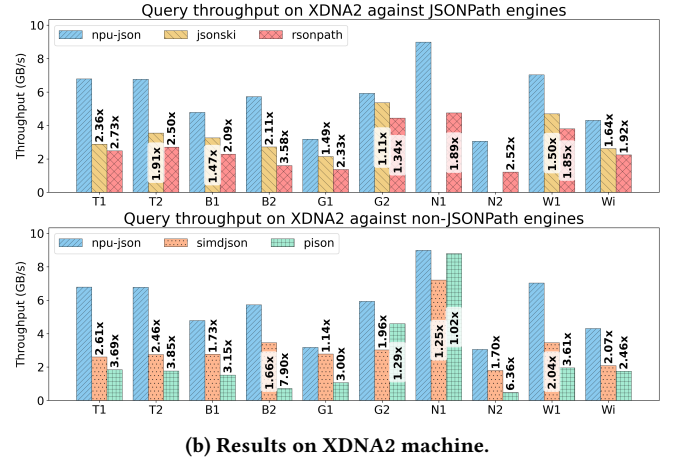
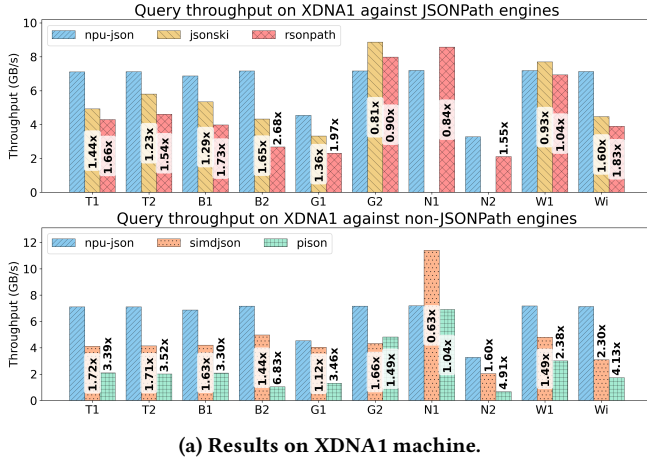


Figure 11: Throughput comparison across the two XDNA platforms. The label on each data point corresponds to the NPU-JSON speedup (e.g., NPU-JSON is 2.38x faster than JSONSki on query T1 running on the XDNA2 machine).

disable turbo boost. For the XDNA1 machine, this corresponds to 4.0 GHz, while for the XDNA2 machine, the maximum clock rate is 2.0 GHz. This setup ensures reproducible and stable performance results across all experiments. As discussed earlier, NPU-JSON can be configured to operate on different chunk sizes by means of a compile-time parameter. Data chunks transferred from the CPU to the NPU should be large enough to allow the NPU to reach its peak performance, but small enough to take as much advantage of pipelining as possible. For all experiments in this section, both machines have been configured to work with the best chunk size (i.e., 8 MB for XDNA2 and 16 MB for XDNA1). In Section 4.3.3 we will discuss the impact of different chunk sizes on the overall performance of NPU-JSON, and we will provide a detailed performance analysis of NPU-JSON internals. Finally, we note that evaluating energy efficiency is not currently possible on AMD XDNA NPUs. At the time of writing, the AMD XDNA software stack exposes no built-in mechanism to read on-chip power for the NPU [7], preventing us from quantifying the impact of NPU offload on overall power consumption.

4.1.1 Baseline JSON analytics libraries. We compare NPU-JSON against state-of-the-art parallel JSON processing libraries in two groups. First, we consider native JSONPath engines: rsonpath [21], a SIMD-based streaming JSONPath engine in Rust, JSONSki [28], a C++ SIMD-based JSONPath engine, and cuJSON [20], a GPU-based JSONPath engine evaluated on the NVIDIA GeForce RTX 4070 available in our XDNA2 machine. These systems provide the most direct comparison to NPU-JSON, since they implement the same query abstraction. We do not compare against Mison [35], as no open-source implementation is available. Second, we compare against high-performance JSON parsers without native JSONPath support: simdjson [34] (v4.2.4), using its on-demand API [30], and Pison [27]. For these systems, we manually implement each query using their APIs. Native JSONPath engines provide feature-equivalent baselines, while parser-based systems approximate the performance of hand-written queries on top of optimized parsers.

4.1.2 Queries and Datasets. We validate NPU-JSON using a variety of queries and real-world datasets. An overview is provided in Table 1. All datasets have been taken unmodified from the benchmarking harnesses used to validate the other JSONPath engines, and are available online [38]. We use the datasets and queries from prior work [28, 47] to ensure comparability with existing evaluations. The selected queries cover a variety of JSONPath features supported by the different systems. We exclude queries that use the descendant operator, as it is not supported by NPU-JSON nor by JSONSki. Together, these queries provide a wide coverage of the common JSONPath query structures. They have a large spread in terms of selectivity, ranging from only 44 to 8,774,410 results. Some queries select data fairly close to the root element, while others nest down through many wildcards.

4.2 JSON Analytics Performance

We first evaluate JSON analytics performance by measuring end-to-end query throughput for the workloads listed in Table 1. We compare our approach against state-of-the-art JSONPath query engines, and then against ad hoc implementations built on top of high-performance JSON parsers.

4.2.1 JSONPath query performance. JSONPath query performance is shown in Figure 11 (top) for both the XDNA1 and XDNA2 machines. We omit the results of query N1 for JSONSki, because JSONSki would skip processing a very large portion of the input JSON, making it an unfair comparison. This query is also omitted in the performance evaluation of rsonpath for the same reason [21].

Performance on XDNA1. When focusing on the results of the XDNA1 architecture, we see that most queries top out at around 6 GB/s. The XDNA1 hardware corresponds to the less powerful NPU variant (four compute columns) paired with the highest clock rate in our study (4.0 GHz). Even under these conditions, which should favor CPU-only SIMD-based systems, NPU-JSON still outperforms other engines in all queries except three (G2, W1 and N1), showing that our NPU-based design can effectively be competitive with state-of-the-art SIMD engines. In particular, NPU-JSON

outperforms JSONSki by 1.26x on average (geomean), with a peak speedup of 1.65x on query B2. It also outperforms rsonpath by 1.48x (geomean), reaching a peak of 2.68x on the same query.

Performance on XDNA2. The XDNA2 architecture has more compute tiles (eight columns) but a CPU with a lower clock frequency (2.0 GHz). The more powerful NPU effectively improves the overall system performance, resulting in significant performance gains despite the lower clock frequency. On XDNA2, NPU-JSON can consistently sustain a higher throughput compared to other JSONPath engines. In particular, NPU-JSON outperforms JSONSki by 1.66x (geomean) with a peak of 2.36x on T1, and outperforms rsonpath by 2.20x (geomean, peak of 3.58x on B2). This result highlights that the performance of NPU-JSON is fundamentally driven by NPU compute capability: with a larger XDNA2 architecture, the NPU-side indexing stages no longer dominate end-to-end throughput, allowing our design to deliver the best performance across all queries even on a lower-frequency CPU.

Interestingly, we also note a slight decrease in throughput for most SIMD libraries on XDNA2. In contrast to the results in [21], we also see that rsonpath is consistently slower than JSONSki on our evaluated hardware. This is possibly due to the lower CPU clock frequency, which offsets the benefits of the newer Zen 5 core despite its improved AVX-512 efficiency. Overall, SIMD-based engines are tightly coupled to CPU frequency and SIMD width, whereas NPU-JSON provides high throughput at much lower CPU clock speeds.

4.2.2 Comparison with simdjson and Pison. After comparing with the existing JSONPath query engines in the previous section, we compare against high-performance JSON parsers, namely simdjson and Pison. Because both systems are general-purpose JSON parsers, they do not support JSONPath queries natively. We therefore implemented each query manually in C++ to obtain the same results using their respective APIs. For simdjson, we use the On-Demand API [30], which allows the parser to employ various skipping techniques similarly to the streaming query engines. The same query logic is implemented on top of Pison to ensure a comparable end-to-end measurement. As can be seen in Figure 11 (bottom row), NPU-JSON has competitive performance and is faster than both simdjson and Pison on most queries. On XDNA1, NPU-JSON outperforms simdjson by 1.46x on average (geomean), with a peak speedup of 2.30x on query Wi. It also outperforms Pison by 3.05x (geomean), reaching a peak of 6.83x on B2. On XDNA2, NPU-JSON outperforms simdjson by 1.81x (geomean, peak of 2.61x on T1), and Pison by 3.09x (geomean, peak of 7.90x on B2).

Compared with rsonpath and JSONSki, simdjson and Pison also decline on XDNA2, though less significantly. Both engines benefit from Zen 5 improved AVX-512 efficiency, partially compensating for the lower clock frequency. Overall, this experiment further confirms that NPU-JSON consistently delivers the highest throughput on both machines, with the exception of a single query where the best SIMD baseline remains slightly faster (i.e., query N1).

4.3 NPU Performance Analysis

The previous section demonstrated that NPU-JSON can outperform or match existing SIMD-based systems. In this section, we analyze the internal properties of our design. The goal is to shed light on

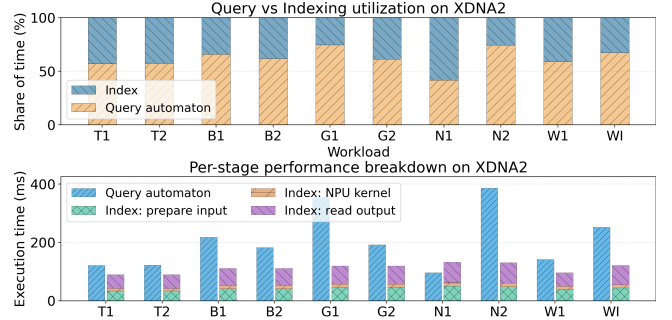


Figure 12: Query automaton (CPU) vs indexing (NPU). Percentage utilization (top) and absolute numbers (bottom).

the performance behavior of NPU-JSON and discuss how the XDNA NPU contributes to JSON processing.

4.3.1 CPU vs NPU Utilization. As discussed earlier, balancing the different performance characteristics of the CPU and NPU can be challenging. To better understand the contribution of the NPU and the CPU to the overall performance, we have instrumented NPU-JSON to measure which of the two components requires the largest fraction of the total execution time. It is important to note that, because of pipelining, both components can execute in parallel, which would make reporting absolute values alone misleading. For this reason, we report both absolute numbers and the pipeline utilization (Figure 12). To compute the latter, we add up the execution times of both the indexing stage on the NPU and the query execution stage on the CPU. We then calculate a percentage of the total execution time for both stages. In an ideal scenario, we would have a 50-50 utilization share between both the CPU and the NPU, which would allow the system to pipeline both stages perfectly. When one component, for example the NPU, takes longer to construct the indexes for a chunk than the CPU can process them, this causes the CPU automaton to wait for the next chunk.

The opposite is also true once the queue of chunks between the components is full: in this case, the NPU must wait to process the next chunk until the CPU automaton can finish the first occupied chunk in the queue. Figure 12 shows the utilization per component for each query in Table 1. This shows a CPU-bound behavior for most workloads: the query automaton accounts for more than half of the traced time in nearly all queries. This is especially visible for G1 and N2, whose deeply nested wildcard-heavy queries require many state transitions and therefore spend the largest fraction of time in CPU query execution. The main exception is N1. This query is shallow and returns only a small number of results, making indexing dominate. Generally, queries with lower query-automaton cost tend to reach the highest XDNA2 throughput, whereas deeply nested or automaton-heavy queries are limited by CPU-side query execution. Even then, NPU-JSON matches or beats SIMD implementations, showing NPU indexing remains effective despite the unbalanced pipeline.

The detailed breakdown (bottom of Figure 12) quantifies the overheads around NPU execution. *Index: prepare input* includes the CPU-side work needed to partition chunks into NPU blocks, construct the metadata required at block boundaries, and arrange

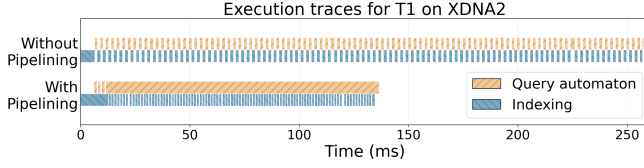


Figure 13: Execution trace for query T1 with and without pipelined execution.

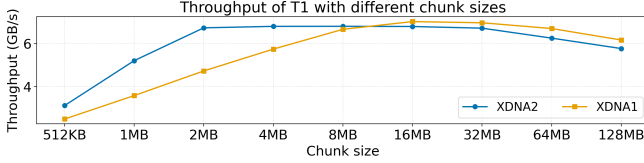


Figure 14: Effect of chunk size on query T1 from Table 1. Other queries have similar behavior.

the input buffers in the format consumed by the XDNA kernels. This preprocessing is included in all end-to-end throughput numbers; it accounts for 37%-40% across the evaluated workloads. *Index: NPU kernel* includes the NPU-specific string-index construction. The kernel time is about 10%, indicating that the core NPU-accelerated workload is small relative to host-side preparation and output transfer.

4.3.2 Impact of Execution Pipelining. To show the effects of this optimization, we have implemented a profiling agent that records the execution traces of each component for each chunk. In Figure 13 we show a Gantt chart visualizing the execution trace for one of the queries in Table 1. The figure shows that pipelining has a significant effect on the end-to-end execution time of the query, more than halving it due to the overlapping execution of the CPU and NPU. Thanks to ping-pong buffering, this means that in the pipelined case, the NPU is busy for nearly the entire duration of the query execution. One other point of interest to note is that the first kernel execution on the NPU takes longer than subsequent executions. This can be seen in Figure 13, but is more apparent at smaller chunk sizes. This is likely because the kernel can be cached in the NPU’s instruction memories. This makes running different kernels in succession less efficient, which is a reason the combined index construction approach was chosen. In Figure 13 we can also see that creating the escape carry index is very fast; with pipelining, we can create it in parallel with chunk processing on the NPU, making it practically free.

4.3.3 Varying Chunk Sizes. As mentioned earlier in Section 3.1, our system can be configured with different chunk sizes, potentially having a significant effect on performance. This is because, at small chunk sizes, the overhead of the external NPU kernel call can become significant due to the increased frequency of kernel calls. Figure 14 shows the effect of chunk size on system performance. Very small chunks have significantly lower performance (although still in the order of GB/s). The performance then increases with larger chunk sizes until a peak is reached at 8 MB for the XDNA2 machine, and 16 MB for the XDNA1 machine. After this, the performance decreases slightly. There are multiple potential reasons

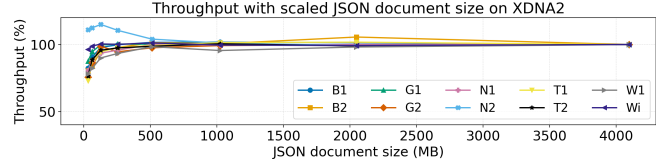


Figure 15: NPU-JSON scalability with different input sizes. Normalized to the throughput with 4 GB objects.

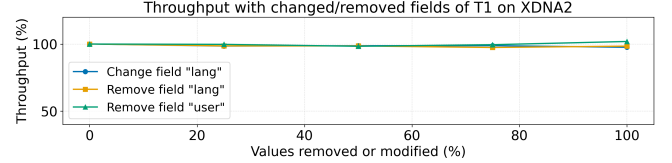


Figure 16: Impact of selectivity on NPU-JSON. Throughput normalized to the max query throughput of query T1.

for this pattern, which unfortunately cannot be directly verified without hardware-level profiling (unavailable on the NPU). First, the larger chunk sizes do not fit into the CPU cache, and therefore when the JSONPath automaton on the CPU is iterating over the large amount of structural characters, cache misses could occur. Another reason is that the potential for pipelining is diminished at larger chunk sizes, due to the first and last chunks being a larger fraction of the total. We also note that in Figure 14, the XDNA2 machine peaks earlier than XDNA1, which needs larger chunks to reach maximum performance. As the XDNA2 architecture has more columns as XDNA1, it should have less data to process on each column for the same chunk size. For this reason one might expect the opposite, as larger chunk sizes are generally faster.

4.3.4 Dataset Scaling. NPU-JSON is designed to process large volumes of JSON data. However, the overhead of using an external NPU accelerator can become significant when the input objects are small. We therefore evaluate the impact of input object size on query performance. For this experiment, we construct smaller input JSON objects for each query across all datasets. Datasets are scaled down by removing fields and values that are not part of the result set, thereby generating inputs of increasing size while preserving the structure relevant to the query. Figure 15 shows the effect of object size on system performance. For very small objects, throughput is noticeably lower. In this case, the cost of transferring data to the accelerator and filling the internal pipelines dominates querying and parsing throughput. As the object size grows, the system becomes fully pipelined and the NPU reaches saturation. At this point, throughput stabilizes, and no further scalability is observed with larger objects, indicating that the accelerator and the query processing pipeline are fully utilized.

4.3.5 Selectivity. We conduct a dedicated experiment to evaluate the effect of query selectivity on system performance. To this end, we varied the selectivity of query T1 by modifying or removing the fields `lang` and `user` in the input JSON documents. This allowed us to control how many elements are returned by the query while keeping the overall structure of the dataset unchanged. Figure 16 presents the performance of our system on the XDNA2 machine.

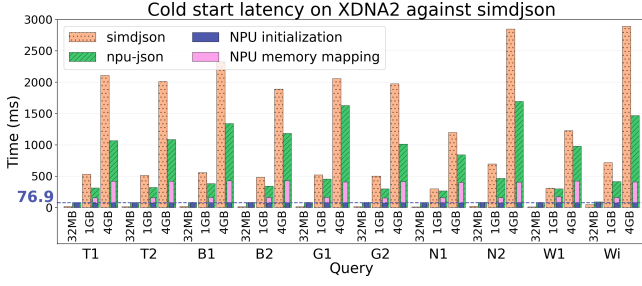


Figure 17: Cold-start query latency (ms) of NPU-JSON compared with simdjson over different input sizes. A breakdown of the dominant first-time setup costs is included.

As the figure shows, query selectivity has no significant impact on system performance. This behavior is expected. The indexing stage on the NPU performs a fixed amount of work per chunk, regardless of the query or its selectivity. The accelerator processes all input data uniformly, so selectivity only affects the amount of result data produced on the CPU. As the overhead of result materialization is relatively small, the overall system throughput remains largely unchanged across different selectivity levels.

4.3.6 Cold Start Performance. Like other accelerators, the NPU incurs a startup cost for initialization and memory mapping before computation can begin. To quantify this overhead, we designed a dedicated cold-start experiment using the same workloads as in Section 4.3.4. We measure the end-to-end latency of executing a single query when the NPU has not been initialized beforehand, excluding file reading, and run each query 30 times from a cold start. Figure 17 reports the geometric mean latency and compares it with simdjson. The figure also breaks down the main sources of cold-start overhead. *NPU Initialization* includes device initialization, hardware-context creation, and internal engine-buffer setup. It is almost constant across queries and input sizes (≈ 76.9 ms). This cost can typically be paid once at application startup and overlapped with other initialization work. *NPU Memory Mapping* maps the input document into an NPU DMA-accessible buffer and scales linearly with the input size (≈ 80 ms/GB in our experiment). We investigated these overheads and traced them to the AMD XDNA driver, and reported the issue to AMD [6]. Overall, the cold-start overhead makes CPU-based execution preferable for small inputs, where the fixed initialization cost cannot be amortized. However, once the input is large enough, the startup cost becomes acceptable, due to the higher throughput from NPU execution.

4.3.7 SIMD-based Indexing Comparison. To isolate the effect of the software design from NPU execution, we implemented an alternative indexing backend using SIMD instructions. This backend preserves the same chunked pipeline, structural-index representation, and JSONPath query automaton of NPU-JSON, but replaces the preprocessing stage, postprocessing stage, and NPU indexing kernels with AVX-512 routines. Comparing this backend against simdjson helps quantify the benefit of our query-engine design, while comparing it against the NPU backend isolates the contribution of NPU indexing acceleration.

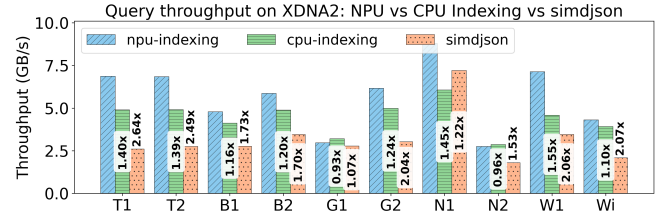


Figure 18: Comparison of the NPU backend, CPU backend, and simdjson on XDNA2.

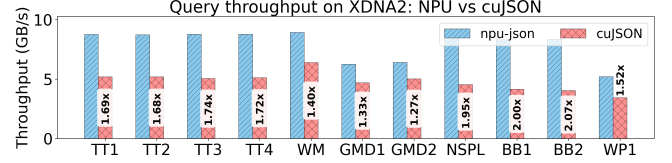


Figure 19: Throughput comparison against cuJSON

Figure 18 shows that the SIMD-based backend is already faster than simdjson on most workloads. This indicates that part of the performance improvement comes from the software organization of NPU-JSON: the query automaton can be pipelined with the indexing engine, exploiting the parallelism of the workload. The NPU backend provides an additional benefit when indexing is a significant part of the total execution time. For example, NPU indexing is 1.45x faster than the CPU backend on N1, 1.40x faster on T1, and 1.55x faster on W1. In contrast, G1 and N2 are heavily dominated by the CPU-side query automaton, as shown in Figure 12. For these workloads, accelerating indexing has little effect on end-to-end throughput, and the CPU backend is close to the NPU backend. This experiment confirms that NPU-based indexing significantly improves structural-index construction.

4.3.8 GPU-based Parsing. Consumer-grade discrete GPUs are often available in machines that also include on-chip NPUs. Our XDNA2 test machine is equipped with an NVIDIA GeForce RTX 4070, allowing us to compare two forms of acceleration available in the same commodity system.

We compare NPU-JSON against cuJSON [20], a state-of-the-art CUDA-based JSONPath engine. cuJSON follows a high-level design similar to NPU-JSON: it constructs structural indexes on the GPU and executes the JSONPath query automaton on the CPU. It therefore provides a relevant baseline for comparing the two implementations on the same commodity hardware. The performance comparison of the two engines is shown in Figure 19. The queries used in this experiment are taken from the cuJSON performance suite in the cuJSON GitHub repository [20] and are used to assess the performance of cuJSON itself. We use this query set, rather than the main workload in Table 1, to ensure that the comparison uses the datasets and queries normally used to evaluate cuJSON. On the evaluated machine and query suite, NPU-JSON outperforms cuJSON across all queries. Our profiling shows that these queries do not significantly stress the CPU-side automaton, indicating that the performance difference mainly arises during structural-index construction. These results demonstrate that the XDNA2 NPU is an

effective accelerator for JSON indexing in the evaluated commodity configuration and is competitive with another form of hardware acceleration available in the same system. These findings do not necessarily extend to server-class GPUs, whose substantially different memory bandwidth, compute resources, and power envelopes may yield different performance.

5 RELATED WORK

SIMD-accelerated JSON parsing on CPUs. JSON documents are typically parsed using single-threaded techniques developed for context-free grammars, such as recursive descent or pushdown automata, which process the input character by character [2]. A foundational work on the use of Single Instruction, Multiple Data (SIMD) instructions to parse JSON is Mison [35]. It breaks from the character-at-a-time model by processing large blocks of input data (e.g., 64 bytes) in parallel using SIMD instructions. Based on Mison, *simdjson* [34] employs a two-stage architecture. In the first stage, it uses bit-parallel algorithms to rapidly construct structural indexes from the raw byte stream. SIMD instructions are used to accelerate this process through vectorization. The result is used in the second stage for validation and navigation. This design allows *simdjson* to achieve gigabytes per second throughput. While *simdjson* advances single-threaded performance, its reliance on global state created data dependencies that prevented straightforward parallelization. This challenge was addressed by Pison [27], which introduced techniques like bitwise rectification to enable fully data-parallel construction of structural indexes across multiple cores. Our work also relies on parallel index construction algorithms. However, we move index construction from the CPU to the NPU, which has a different programming model and performance characteristics, requiring re-designed algorithms for both computation as well as state and memory management.

Streaming JSONPath Query Engines. Parsing the complete document is unnecessary for many analytics workloads. This has led to the development of streaming query engines that fuse parsing with query evaluation. Engines like *JSONSki* [28] and *rsonpath* [21] leverage structural indexes to enable query-aware skipping. Instead of building a full parse tree, they operate a state machine that can fast-forward over large, irrelevant sections of the document. For instance, when searching for a specific key within an object, if the engine encounters a nested object or array that does not match the query path, parsing can be skipped. After finding a target field, the engine can also skip to the end of the current object and start searching within the next object. *rsonpath* further optimizes this model for descendant queries (*.*), which search for elements at any depth. It enables even more aggressive skipping, as the engine can bypass entire subtrees while searching for the next potential match. A similar concept of fast-forwarding with structural indexes is adopted in *NPU-JSON*. We offload the data-parallel indexing stage entirely to the NPU, freeing the CPU to focus solely on executing the query automaton, enabling pipelined, parallel execution between the two processors. This separation of concerns allows us to exploit the NPU’s capabilities for indexing while leveraging the CPU in complex control flow for query evaluation.

Hardware accelerated JSON parsing. Parallel JSON parsing has also been explored on FPGAs and GPUs. *PipeJSON* [15] applies

the *simdjson* structural index construction onto a custom FPGA and pipelines many procedures like string indexing and number parsing to exploit parallelism. It is able to parse JSON at the speed of the PCIe v3 bandwidth. *SPEAR-JSON* [24] implements an FPGA parser architecture that is able to extract attributes based on a query expression using fewer FPGA resources than existing techniques. Peltenburg et al. [39] identify the parsing bottleneck in JSON-to-Arrow conversion. They present a custom FPGA accelerator, improving throughput by 3 to 6 times and energy efficiency by 14 to 59 times over a *RapidJSON*-based software baseline. *GpJSON* [47] and *cJSON* [20] adopt structural indexing on the GPU (via CUDA) to execute JSONPath queries. The structural index construction strategies are adjusted to allow for massive parallelization, enabling both engines to saturate thousands of threads on the GPU. Targeting the AMD XDNA NPU presents a distinct set of challenges compared with these platforms. Unlike the globally shared memory available on GPUs, XDNA uses strictly distributed and size-limited local memories together with a dataflow execution model, requiring a different approach to data tiling and state management. A comparison against *cJSON* is included in Section 4.3.8.

NPU for general-purpose computing. Recent work has begun to explore the use of NPUs for general-purpose computing. *TINA* [11] is a framework to implement non Neural Network (NN) signal processing workloads on NN accelerators (e.g., GPUs, and TPUs), by mapping operations to NN layers, including convolutional and fully connected layers. On AMD NPUs and, more generally, AIEs, prior studies focus on scientific and compute-intensive applications: van Wijhe et al. use the AI Engine for real-time digital signal processing in radio astronomy [50]; different solutions have been proposed to accelerate stencil computations [12, 46, 53]; Klaisoongnoen et al. used AIEs for market analysis [31]; Menzel and Plessl ported the electron repulsion integrals computation [37]. Our work complements these efforts by targeting JSON analytics, where data-parallel structural indexing is coupled with branch-heavy query execution.

6 CONCLUSION

In this paper, we demonstrated that widely available AMD XDNA NPUs can accelerate a non-AI data-processing workload such as JSON parsing. We introduced *NPU-JSON*, an open-source JSONPath query engine that offloads structural indexing to the XDNA NPU while executing JSONPath queries on the CPU. Across the evaluated systems and workloads, this hybrid design achieves performance competitive with state-of-the-art SIMD-based engines. Tile-level NPU parallelism and CPU-NPU pipelining are crucial to obtaining this performance.

In addition to the end-to-end results, our experiments characterize important performance properties of two generations of AMD XDNA NPUs, including bandwidth limits, vector-kernel efficiency, and predictable execution without branch prediction or instruction-level parallelism. These findings motivate exploring hybrid CPU-NPU designs for data-processing workloads beyond JSON parsing. Workloads that combine data-parallel stages with branch-heavy CPU processing are especially promising candidates.

REFERENCES

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Berkeley, CA, USA, 265–283.
- [2] A.V. Aho, R. Sethi, and J.D. Ullman. 1986. *Compilers, Principles, Techniques, and Tools*. Addison-Wesley Publishing Company, Reading, MA, USA.
- [3] AMD. 2020. *AMD64 Architecture Programmer's Manual*. Advanced Micro Devices, Inc. https://docs.amd.com/v/u/en-US/40332-PUB_4.08 Accessed: 2026-06-09.
- [4] AMD. 2023. *On-Chip AI Integration is the Future of PC Computing*. Advanced Micro Devices, Inc. <https://www.amd.com/en/blogs/2023/on-chip-ai-integration-is-the-future-of-pc-computing>. Accessed: 2026-06-09.
- [5] AMD. 2025. *Xilinx® Runtime (XRT) Architecture*. Advanced Micro Devices, Inc. <https://xilinx.github.io/XRT/2025.1/html/index.html> Software documentation for the AMD Xilinx runtime library. Accessed: 2026-06-09.
- [6] AMD XDNA GitHub. 2026. *GitHub Issue #1399: xrtBOAAllocUserPTR scales with 4K page count even for hugepage-backed buffers*. GitHub. <https://github.com/amd/xdna-driver/issues/1399> GitHub issue to the AMD development team (perma-link), reporting the source of cold-start overhead in the NPU driver. Accessed: 2026-06-09.
- [7] AMD XDNA GitHub. 2026. *GitHub Issue #713: Support for power reading*. GitHub. <https://github.com/amd/xdna-driver/issues/713#issuecomment-3885803735> GitHub issue with our request to the AMD development team (perma-link), and confirmation that power reading is currently not possible as it depends on a missing kernel feature. Accessed: 2026-06-09.
- [8] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. <https://doi.org/10.1145/3620665.3640366>
- [9] Apple Inc. 2026. Apple Core ML. <https://developer.apple.com/documentation/CoreML> Accessed: 2026-06-09.
- [10] Alan Baradlay, Antti Koivisto, Matt Woodrow, Patrick Angle, Ryosuke Niwa, Vitor Ribeiro Roriz, Wenson Hsieh, and Yusuke Suzuki. 2024. *Optimizing WebKit and Safari for Speedometer 3.0*. WebKit Project. <https://webkit.org/blog/15249/optimizing-webkit-safari-for-speedometer-3-0/> Accessed: 2026-06-09.
- [11] Christiaan Boerkamp, Steven Van der Vlugt, and Zaid Al-Ars. 2024. TINA: Acceleration of Non-NN Signal Processing Algorithms Using NN Accelerators. In *2024 IEEE 34th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, Piscataway, NJ, USA, 1–6. <https://doi.org/10.1109/MLSP58920.2024.10734727>
- [12] Nick Brown. 2023. Exploring the Versal AI Engines for Accelerating Stencil-based Atmospheric Advection Simulation. In *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (Monterey, CA, USA) (FPGA '23)*. Association for Computing Machinery, New York, NY, USA, 91–97. <https://doi.org/10.1145/3543622.3573047>
- [13] H. Butler, M. Daly, A. Doyle, Sean Gillies, S. Hagen, and T. Schaub. 2016. The GeoJSON Format. RFC 7946. <https://doi.org/10.17487/RFC7946>
- [14] ByteDance. 2025. Introduction to Sonic. <https://github.com/bytedance/sonic/blob/b6510a9ec655f0649a7d587131f535d635a50524/docs/INTRODUCTION.md> Accessed: 2026-06-09.
- [15] Jonas Dann, Royden Wagner, Daniel Ritter, Christian Faerber, and Holger Froening. 2022. PipeJSON: Parsing JSON at Line Speed on FPGAs. In *Proceedings of the 18th International Workshop on Data Management on New Hardware (Philadelphia, PA, USA) (DaMoN '22)*. Association for Computing Machinery, New York, NY, USA, Article 3, 7 pages. <https://doi.org/10.1145/3533737.3535094>
- [16] Databricks, Inc. 2025. Databricks Lakehouse Platform. <https://www.databricks.com/product/data-lakehouse> Accessed: 2026-06-09.
- [17] Elastic NV. 2025. Elasticsearch, Logstash, Kibana (ELK Stack). <https://www.elastic.co/what-is/elk-stack> Accessed: 2026-06-09.
- [18] Joseph A. Fisher. 1983. Very Long Instruction Word architectures and the ELI-512. *SIGARCH Comput. Archit. News* 11, 3 (June 1983), 140–150. <https://doi.org/10.1145/1067651.801649>
- [19] Brian Gaide, Dinesh Gaitonde, Chirag Ravishankar, and Trevor Bauer. 2019. Xilinx Adaptive Compute Acceleration Platform: Versal™ Architecture. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '19)*. ACM, New York, NY, USA, 84–93. <https://doi.org/10.1145/3289602.3293906>
- [20] Ashkan Vedadi Gargary, Soroosh Safari Loaliyan, and Zhijia Zhao. 2025. cuJSON: A Highly Parallel JSON Parser for GPUs. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 85–100. <https://doi.org/10.1145/3760250.3762222>
- [21] Mateusz Gienieccko, Filip Murlak, and Charles Paperman. 2023. Supporting Descendants in SIMD-Accelerated JSONPath. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4 (Vancouver, BC, Canada) (ASPLOS '23)*. Association for Computing Machinery, New York, NY, USA, 338–361. <https://doi.org/10.1145/3623278.3624754>
- [22] Stefan Gössner. 2007. JSONPath - XPath for JSON. <https://goessner.net/articles/JsonPath/> Accessed: 2026-06-09.
- [23] Stefan Gössner, Glyn Normington, and Carsten Bormann. 2024. JSONPath: Query Expressions for JSON. RFC 9535. <https://doi.org/10.17487/RFC9535>
- [24] Tobias Hahn, Stefan Wildermann, and Jürgen Teich. 2023. SPEAR-JSON: Selective Parsing of JSON to Enable Accelerated Stream Processing on FPGAs. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, Piscataway, NJ, USA, 189–196. <https://doi.org/10.1109/FPL60245.2023.00034>
- [25] Erika Hunhoff, Joseph Melber, Kristof Denolf, Andra Bisca, Samuel Bayliss, Stephen Neuendorffer, Jeff Fifield, Jack Lo, Pranathi Vasireddy, Phil James-Roxby, and Eric Keller. 2025. Efficiency, Expressivity, and Extensibility in a Close-to-Metal NPU Programming Interface. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Piscataway, NJ, USA, 85–94. <https://doi.org/10.1109/FCCM62733.2025.00043>
- [26] Intel Corporation. 2026. *Intel® Distribution of OpenVINO™ Toolkit (Overview)*. Intel. <https://www.intel.com/content/www/us/en/developer/tools/openvino-toolkit/overview.html> Accessed: 2026-06-09.
- [27] Lin Jiang, Junqiao Qiu, and Zhijia Zhao. 2020. Scalable structural index construction for JSON analytics. *Proc. VLDB Endow.* 14, 4 (Dec. 2020), 694–707. <https://doi.org/10.14778/3436905.3436926>
- [28] Lin Jiang and Zhijia Zhao. 2022. JSONski: streaming semi-structured data with bit-parallel fast-forwarding. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 200–211. <https://doi.org/10.1145/3503222.3507719>
- [29] JSON.org. 2026. Introducing JSON. <https://www.json.org/json-en.html> Accessed: 2026-06-09.
- [30] John Keiser and Daniel Lemire. 2024. On-demand JSON: A better way to parse documents? *Software: Practice and Experience* 54, 6 (2024), 1074–1086. <https://doi.org/10.1002/spe.3313>
- [31] Mark Klaisoonognoen, Nick Brown, Tim Dykes, Jessica R. Jones, and Utz-Uwe Haus. 2024. Evaluating Versal AI Engines for Option Price Discovery in Market Risk Analysis. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (Monterey, CA, USA) (FPGA '24)*. Association for Computing Machinery, New York, NY, USA, 176–182. <https://doi.org/10.1145/3626202.3637578>
- [32] Adam Klein. 2019. *V8 Release v7.6*. V8 Project. <https://v8.dev/blog/v8-release-76> Accessed: 2026-06-09.
- [33] Chester Lam. 2025. Zen 5's AVX-512 Frequency Behavior. <https://chipsandcheese.com/p/zen-5s-avx-512-frequency-behavior> Accessed: 2026-06-09.
- [34] Geoff Langdale and Daniel Lemire. 2019. Parsing Gigabytes of JSON per Second. *The VLDB Journal* 28, 6 (2019), 941–960. <https://doi.org/10.1007/s00778-019-00578-5>
- [35] Yinan Li, Nikos R. Katsipoulakis, Badrish Chandramouli, Jonathan Goldstein, and Donald Kossmann. 2017. Mison: a fast JSON parser for data analytics. *Proc. VLDB Endow.* 10, 10 (June 2017), 1118–1129. <https://doi.org/10.14778/3115404.3115416>
- [36] Sean Lie. 2021. Multi-Million Core, Multi-Wafer AI Cluster. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 1–41. <https://doi.org/10.1109/HCS52781.2021.9567153>
- [37] Johannes Menzel and Christian Plessl. 2025. Efficient and Distributed Computation of Electron Repulsion Integrals on AMD AI Engines. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Piscataway, NJ, USA, 95–104. <https://doi.org/10.1109/FCCM62733.2025.00044>
- [38] Charles Paperman, Mateusz Gienieccko, and Filip Murlak. 2023. Dataset for benchmarking rsonpath. <https://doi.org/10.5281/zenodo.8386388> Accessed: 2026-06-09.

- [39] Johan Peltenburg, Akos Hadnagy, Matthijs Brobbel, Robert Morrow, and Zaid Al-Ars. 2021. Tens of gigabytes per second JSON-to-Arrow conversion with FPGA accelerators. In *2021 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, Piscataway, NJ, USA, 1–9. <https://doi.org/10.1109/ICFPT52863.2021.9609833>
- [40] PostgreSQL. 2026. *PostgreSQL 17.10 Documentation*. The PostgreSQL Global Development Group. <https://www.postgresql.org/docs/17/datatype-json.html> Accessed: 2026-06-09.
- [41] Qualcomm Technologies, Inc. 2026. *Qualcomm Neural Processing SDK for AI*. Qualcomm Developer. <https://www.qualcomm.com/developer/software/neural-processing-sdk-for-ai> Accessed: 2026-06-09.
- [42] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [43] RFC9535. 2024. *JSONPath specifications: 2.4. Function Extensions*. <https://www.rfc-editor.org/rfc/rfc9535.html#name-function-extensions>
- [44] Alejandro Rico, Satyaprakash Pareek, Javier Cabezas, David Clarke, Baris Ozgul, Francisco Barat, Yao Fu, Stephan Münz, Dylan Stuart, Patrick Schlangen, Pedro Duarte, Sneha Date, Indrani Paul, Jian Weng, Sonal Santan, Vinod Kathail, Ashish Sirasao, and Juanjo Noguera. 2024. AMD XDNA NPU in Ryzen AI Processors. *IEEE Micro* 44, 6 (Nov. 2024), 73–82. <https://doi.org/10.1109/MM.2024.3423692>
- [45] GitHub simdjson project. 2019. *Parse file larger than 4 GB (Issue #128)*. GitHub. <https://github.com/simdjson/simdjson/issues/128> Issue tracker entry for the simdjson project. Accessed: 2026-06-09.
- [46] Gagandeep Singh, Alireza Khodamoradi, Kristof Denolf, Jack Lo, Juan Gomez-Luna, Joseph Melber, Andra Bisca, Henk Corporaal, and Onur Mutlu. 2023. SPARTA: Spatial Acceleration for Efficient and Scalable Horizontal Diffusion Weather Stencil Computation. In *Proceedings of the 37th ACM International Conference on Supercomputing (Orlando, FL, USA) (ICS '23)*. Association for Computing Machinery, New York, NY, USA, 463–476. <https://doi.org/10.1145/3577193.3593719>
- [47] Sacheendra Talluri, Guido Walter Di Donato, Luca Danelutti, Koen R. Vlaswinkel, Marco Arnaboldi, Arnaud Delamare, Marco D. Santambrogio, and Daniele Bonetta. 2025. GpJSON: High-Performance JSON Data Processing on GPUs. *Proc. VLDB Endow.* 18, 9 (May 2025), 3216–3229. <https://doi.org/10.14778/3746405.3746439>
- [48] RAPIDS Development Team. 2023. *RAPIDS: Libraries for End to End GPU Data Science*. NVIDIA. <https://rapids.ai> Accessed: 2026-06-09.
- [49] Tenstorrent. [n.d.]. Blackhole Cards. <https://tenstorrent.com/en/hardware/cards>. Accessed: 2026-06-09.
- [50] Victor Van Wijhe, Vincent Sprave, Daniele Passaretti, Nikolaos Alachiotis, Gerit Grutzeck, Thilo Pionteck, and Steven Van der Vlugt. 2024. Exploring the Versal AI Engines for Signal Processing in Radio Astronomy. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, Piscataway, NJ, USA, 1–10. <https://doi.org/10.1109/FPL64840.2024.00011>
- [51] AMD Xilinx. 2022-2024. *Versal Adaptive SoC AIE-ML Architecture Manual (AM020)*. Advanced Micro Devices, Inc. AMD. <https://docs.amd.com/r/en-US/am020-versal-aie-ml> Accessed: 2026-06-09.
- [52] AMD Xilinx. 2024. *AI Engine API User Guide*. Advanced Micro Devices, Inc. AMD. https://www.xilinx.com/htmldocs/xilinx2024_1/aiengine_api/aiengine_api/doc/index.html Accessed: 2026-06-09.
- [53] Zhenyu Xu, Miaoxiang Yu, Yazhe Zhang, Jillian Cai, Qing Yang, and Tao Wei. 2025. Tile-Level Pipeline for Linear Scalable Stencil Computation on AMD AI Engines. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (Monterey, CA, USA) (FPGA '25)*. Association for Computing Machinery, New York, NY, USA, 172–178. <https://doi.org/10.1145/3706628.3708822>